

جلسه 3

نوع داده لیست در پایتون

پایتون، طیف وسیعی از نوع داده‌های ترکیبی را فراهم می‌کند که معمولاً به عنوان یک دنباله به آن‌ها ارجاع داده می‌شود. لیست یکی از پرکاربردترین و متنوع‌ترین انواع عددی موجود در پایتون است.

روش ساخت نوع داده لیست در پایتون

در برنامه‌نویسی پایتون، نوع داده لیست با قرار دادن همه آیتم‌ها (عناصر) درون یک براکت (کمانک) یعنی `[]`، ساخته می‌شود. عناصر یک لیست، با استفاده از علامت ویرگول، یعنی «`,`»، از هم جدا می‌شوند.

```
1 # empty list
2 my_list = []
3
4 # List of integers
5 my_list = [1, 2, 3]
6
7 # List with mixed datatypes
8 my_list = [1, "Hello", 3.4]
```

روش دسترسی داشتن به عناصر یک لیست

اندیس لیست

می‌توان از عملگر «اندیس» (Operator) یعنی `[]`، برای دسترسی داشتن به یک عنصر از لیست، استفاده کرد. اندیس لیست‌ها در پایتون از عدد ۰ شروع می‌شود.

اندیس‌دهی منفی

پایتون، امکان اندیس‌دهی منفی برای یک توالی را نیز فراهم می‌کند. در مثالی که در ادامه ارائه شده است، اندیس ۱- به آخرین عنصر از لیست و ۲- به عنصر یکی مانده به آخر (دومین عنصر از آخر لیست به اول آن) اشاره دارد و به همین ترتیب می‌توان با اندیس‌دهی منفی، عناصر لیست را از آخر به اول فراخوانی کرد.

```

1 my_list = ['p','r','o','b','e']
2 # Output: p
3 print(my_list[0])
4
5 # Output: o
6 print(my_list[2])
7
8 # Output: e
9 print(my_list[4])
10
11 # Error! Only integer can be used for indexing
12 # my_list[4.0]
13
14 # Nested List
15 n_list = ["Happy", [2,0,1,5]]
16
17 # Nested indexing
18
19 # Output: a
20 print(n_list[0][1])
21
22 # Output: 5
23 print(n_list[1][3])

```

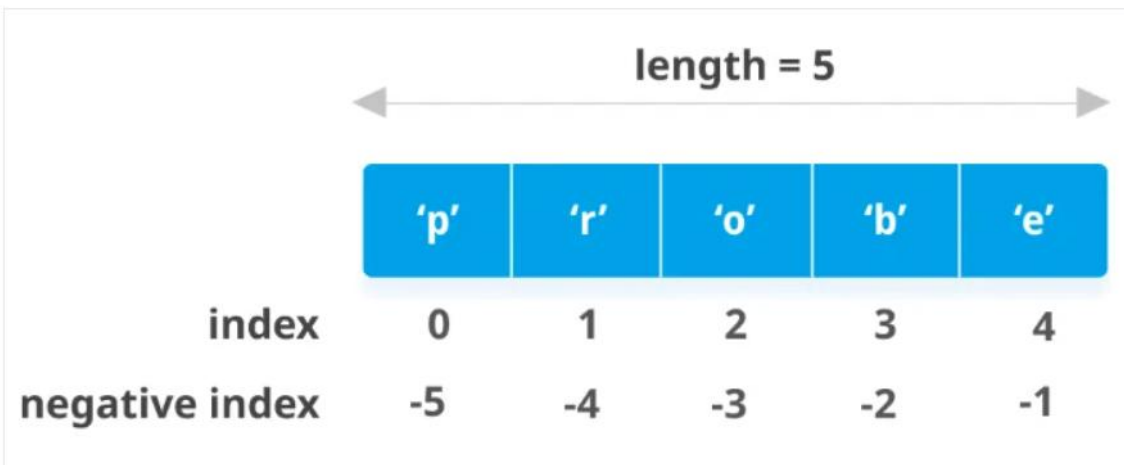
اندیس‌دهی منفی

پایتون، امکان اندیس‌دهی منفی برای یک توالی را نیز فراهم می‌کند. در مثالی که در ادامه ارائه شده است، اندیس ۱- به آخرین عنصر از لیست و ۲- به عنصر یکی مانده به آخر (دومین عنصر از آخر لیست به اول آن) اشاره دارد و به همین ترتیب می‌توان با اندیس‌دهی منفی، عناصر لیست را از آخر به اول فراخوانی کرد.

```

1 my_list = ['p','r','o','b','e']
2
3 # Output: e
4 print(my_list[-1])
5
6 # Output: p
7 print(my_list[-5])

```



روش برش زدن لیست در پایتون

می‌توان به طیف وسیعی از عناصر لیست با استفاده از عملگر برش زدن یعنی «دونقطه (colon)» دسترسی داشت.

```
1 my_list = ['p','r','o','g','r','a','m','i','z']
2 # elements 3rd to 5th
3 print(my_list[2:5])
4
5 # elements beginning to 4th
6 print(my_list[:5])
7
8 # elements 6th to end
9 print(my_list[5:])
10
11 # elements beginning to end
12 print(my_list[:])
```

روش تغییر عنصر کنونی و یا درج عنصر جدید در لیست

نوع داده لیست در پایتون قابل تغییر محسوب می‌شود. شایان ذکر است، نوع داده‌های «تاپل (Tuple)» و «رشته (String)» در پایتون غیر قابل تغییر هستند. اما باید توجه کنید که بین این نوع‌های داده مختلف تفاوت‌های مهمی وجود دارد. برای مثال، برنامه نویسان پایتون باید تفاوت‌های بین لیست و تاپل در پایتون را بلد باشند.

```
1 # mistake values
2 odd = [2, 4, 6, 8]
3
4 # change the 1st item
5 odd[0] = 1
6
7 # Output: [1, 4, 6, 8]
8 print(odd)
9
10 # change 2nd to 4th items
11 odd[1:4] = [3, 5, 7]
12
13 # Output: [1, 3, 5, 7]
14 print(odd)
```

می‌توان «یک عنصر» را با استفاده از متد (append) و «چند عنصر» را با استفاده از متد (extend) به لیست اضافه کرد.

```
1 odd = [1, 3, 5]
2
3 odd.append(7)
4
5 # Output: [1, 3, 5, 7]
6 print(odd)
7
8 odd.extend([9, 11, 13])
9
10 # Output: [1, 3, 5, 7, 9, 11, 13]
11 print(odd)
```

روش حذف یک عنصر از لیست

می‌توان یکی یا تعداد بیشتری از عناصر یک لیست را با استفاده از کلیدواژه `del` حذف کرد.

```
1 my_list = ['p','r','o','b','l','e','m']
2
3 # delete one item
4 del my_list[2]
5
6 # Output: ['p', 'r', 'b', 'l', 'e', 'm']
7 print(my_list)
8
9 # delete multiple items
10 del my_list[1:5]
11
12 # Output: ['p', 'm']
13 print(my_list)
14
15 # delete entire list
16 del my_list
17
18 # Error: List not defined
19 print(my_list)
```

متد pop() عنصر را حذف و آخرین عنصر را در صورتی که اندیسی داده نشده باشد، باز می‌گرداند. این کار کمک می‌کند تا بتوان لیست‌ها را به عنوان پشته پیاده‌سازی کرد (اولین ورودی، آخرین خروجی). می‌توان از متد clear() نیز برای خالی کردن لیست استفاده کرد.

```
1 my_list = ['p','r','o','b','l','e','m']
2 my_list.remove('p')
3
4 # Output: ['r', 'o', 'b', 'l', 'e', 'm']
5 print(my_list)
6
7 # Output: 'o'
8 print(my_list.pop(1))
9
10 # Output: ['r', 'b', 'l', 'e', 'm']
11 print(my_list)
12
13 # Output: 'm'
14 print(my_list.pop())
15
16 # Output: ['r', 'b', 'l', 'e']
17 print(my_list)
18
19 my_list.clear()
20
21 # Output: []
22 print(my_list)
```

برخی از متدهای مورد استفاده در لیست list

- `append()` یک عنصر را به انتهای لیست اضافه می‌کند.
- `extend()` همه عناصر لیست را به لیست دیگری اضافه می‌کند.
- `insert()` یک عنصر را در اندیس تعریف شده درج می‌کند.
- `remove()` یک عنصر را از لیست حذف می‌کند.
- `pop()` یک عنصر را در اندیس داده شده حذف می‌کند و مقدار آن را باز می‌گرداند.
- `clear()` همه عناصر را از لیست حذف می‌کند.
- `index()` اندیس اولین عنصر مطابق با آرگومان متد را باز می‌گرداند.
- `count()` تعداد عناصری که مطابق با آرگومان داده شده به متد هستند را باز می‌گرداند.
- `sort()` عناصر موجود در لیست را به ترتیب صعودی مرتب می‌کند.
- `reverse()` ترتیب عناصر موجود در لیست را معکوس می‌کند.
- `copy()` یک Shallow Copy از لیست را باز می‌گرداند.

کارکرد هر یک از این توابع در لیست:

- `enumerate()` یک شی شمارشی را باز می‌گرداند. شامل اندیس و مقدار همه عناصر یک لیست به صورت یک «تاپل (tuple)» است.
- `len()` طول لیست (تعداد عناصر لیست) را باز می‌گرداند.
- `list()` یک نوع تکرار شونده (تاپل، رشته، مجموعه یا دیکشنری) را به لیست تبدیل می‌کند.
- `max()` بزرگترین عنصر لیست را باز می‌گرداند.
- `min()` کوچکترین عنصر لیست را باز می‌گرداند.
- `sorted()` یک لیست مرتب شده جدید را باز می‌گرداند (خود لیست را مرتب نمی‌کند).
- `sum()` مجموع همه عناصر موجود در لیست را باز می‌گرداند.

ساختار داده یا همان ساختمان داده یکی از مفاهیم اصلی و مهم در برنامه نویسی است که افراد علاقه‌مند به یادگیری این حوزه، در ابتدای مسیر آموزش خود با آن آشنا می‌شوند. از ساختار داده در برنامه نویسی می‌توان به منظور ذخیره‌سازی و سازمان‌دهی داده‌ها استفاده کرد. هر ساختار داده در زبان‌های برنامه نویسی، روشی خاص و منحصر به فردی را برای ذخیره کردن داده و دسترسی به آن‌ها فراهم می‌کند.

در زبان برنامه نویسی پایتون، چهار نوع ساختار داده با نام‌های «لیست (List)»، «تاپل (Tuple)»، «مجموعه (Set)» و «دیکشنری (Dictionary)» وجود دارد.

دیکشنری در پایتون

دیکشنری در پایتون یکی از ساختارهای داده مهم و پرکاربرد به حساب می‌آید. این ساختار داده مشابه ساختارهای داده در سایر زبان‌های برنامه نویسی است که از آن‌ها برای نگاشت «کلید (Key)» به «مقدار (Value)» استفاده می‌شود.

ویژگی‌های دیکشنری در پایتون چیست ؟

- دیکشنری در پایتون برخلاف ساختار داده لیست، از ترتیب خاصی برای ذخیره کردن داده‌ها استفاده نمی‌کند.
- با استفاده از کلیدهای دیکشنری، می‌توان مقادیر آن‌ها را بازیابی کرد.
- برای تعریف کلیدها می‌توان از اعداد صحیح و اعشاری، رشته و ساختار داده تاپل استفاده کرد.
- برای تعریف کلیدها در پایتون از ساختار داده دیکشنری و لیست استفاده نمی‌شود.
- کلیدهای دیکشنری پایتون باید منحصر بفرد باشند. بنابراین، از عبارت‌های تکراری نمی‌توان برای ساخت کلیدها استفاده کرد.
- آیتم‌های تکراری را می‌توان برای مقادیر کلیدهای دیکشنری در پایتون به کار برد.
- مقادیر کلیدها می‌توانند از هر نوع داده و ساختار داده‌ای تعریف شوند.
- یکی از ویژگی‌های دیکشنری در پایتون، «قابل تغییر بودن (Mutable)» آن است؛ به عبارتی، پس از تعریف و مقداردهی اولیه آن، می‌توان در طول برنامه، علاوه بر درج آیتم جدیدی به آن، مقداری را به‌روزرسانی یا حذف کرد.
- دیکشنری در پایتون امکان ساخت دیکشنری‌های تو در تو را فراهم می‌کند.
- در زبان پایتون می‌توان ساختارهای داده را به یکدیگر تبدیل کرد. به عبارتی، هر یک از ساختارهای داده در پایتون، ویژگی‌های خاصی دارند که برنامه‌نویسان بر حسب نیاز خود می‌توانند اطلاعات خود را از یک ساختار داده به ساختار داده دیگری منتقل کنند. ساختار داده دیکشنری نیز از این قاعده مستثنی نیست. به عنوان مثال، روش‌های مختلفی برای تبدیل دیکشنری به لیست و لیست به دیکشنری وجود دارند که می‌توان به‌سادگی از آن‌ها بهره برد.

علامت آکولاد برای ساخت دیکشنری

برای تعریف دیکشنری در پایتون از علامت آکولاد ({}) و برای نگاشت کلیدها به مقادیر از علامت دو نقطه (:) استفاده می‌شود. در ادامه، «قاعده نحوی» (سینتکس) پایتون برای ایجاد دیکشنری ارائه شده است.

```
1 MLB_team = {  
2     'Colorado' : 'Rockies',  
3     'Boston'   : 'Red Sox',  
4     'Minnesota': 'Twins',  
5     'Milwaukee': 'Brewers',  
6     'Seattle'  : 'Mariners'  
7 }
```

دسترسی به مقادیر دیکشنری در پایتون

پس از ذخیره آیتم‌های کلید-مقدار در دیکشنری پایتون، می‌توان با استفاده از کلیدها، به مقادیرشان دسترسی داشت. دو روش برای دسترسی به مقادیر کلیدهای دیکشنری پایتون وجود دارد که در ادامه ملاحظه می‌شوند:

- استفاده از کلیدهای دیکشنری
- استفاده از متد `get` دیکشنری

استفاده از نام کلید برای دسترسی به مقادیر دیکشنری

```
1 MLB_team = dict([  
2     ('Colorado', 'Rockies'),  
3     ('Boston', 'Red Sox'),  
4     ('Minnesota', 'Twins'),  
5     ('Milwaukee', 'Brewers'),  
6     ('Seattle', 'Mariners')  
7 ])  
8  
9 MLB_team['Minnesota']
```

خروجی قطعه کد فوق در ادامه ملاحظه می‌شود.

```
'Twins'
```

استفاده از متد Get برای دسترسی به مقادیر دیکشنری

از متد `get` نیز به منظور بازگرداندن مقدار کلید مورد نظر استفاده می‌شود. این متد دارای دو پارامتر است. اولین پارامتر، نام کلیدی را مشخص می‌کند که مقدار آن باید در خروجی بازگردانده شود. دومین پارامتر متد `get`، مقدار پیش‌فرضی است که می‌توان برای کلید در نظر گرفت. به عبارتی، چنانچه کلید مورد نظر در دیکشنری یافت نشد، کامپایلر به جای خطا، مقدار تعیین شده برای پارامتر دوم این متد را برمی‌گرداند. به‌طور پیش‌فرض، مقدار این پارامتر برابر با `None` است. در قطعه کد زیر، مثالی از نحوه کاربرد این متد در دیکشنری پایتون ارائه شده است.

```
1 d = {'a': 10, 'b': 20, 'c': 30}
2
3 print(d.get('b'))
4
5 print(d.get('z'))
6
7 print(d.get('z', -1))
```

خروجی قطعه کد فوق در ادامه ملاحظه می‌شود.

```
20
None
-1
```

ویرایش دیکشنری در پایتون

به منظور ویرایش مقادیر دیکشنری، می‌توان با تخصیص مقادیر جدید به کلیدهایی که از قبل در دیکشنری وجود داشته‌اند، مقادیر آن‌ها را تغییر داد. در قطعه کد زیر، مثالی از ویرایش مقدار مربوط به کلید Seattle از دیکشنری MLB_team ارائه شده است.

```
1 MLB_team = dict([
2     ('Colorado', 'Rockies'),
3     ('Boston', 'Red Sox'),
4     ('Minnesota', 'Twins'),
5     ('Milwaukee', 'Brewers'),
6     ('Seattle', 'Mariners')
7 ])
8
9 MLB_team['Seattle'] = 'Seahawks'
10 print(MLB_team)
```

خروجی قطعه کد فوق در ادامه ملاحظه می‌شود.

```
{'Colorado': 'Rockies', 'Boston': 'Red Sox', 'Minnesota': 'Twins',
'Milwaukee': 'Brewers', 'Seattle': 'Seahawks', 'Kansas City': 'Royals'}
```

حذف مقادیر از دیکشنری در پایتون

چنانچه نیازی به آیتمی خاص در دیکشنری پایتون نباشد، می‌توان جفت کلید-مقدار را با دستور `del` از دیکشنری حذف کرد. در قطعه کد زیر، مثالی از نحوه حذف کلید `Seattle` و مقدار آن از دیکشنری `MLB_team` ارائه شده است.

```
1 MLB_team = dict([
2     ('Colorado', 'Rockies'),
3     ('Boston', 'Red Sox'),
4     ('Minnesota', 'Twins'),
5     ('Milwaukee', 'Brewers'),
6     ('Seattle', 'Mariners')
7 ])
8
9 del MLB_team['Seattle']
10 print(MLB_team)
```

خروجی قطعه کد فوق در ادامه ملاحظه می‌شود.

```
{'Colorado': 'Rockies', 'Boston': 'Red Sox', 'Minnesota': 'Twins',
'Milwaukee': 'Brewers', 'Kansas City': 'Royals'}
```

متدهای دیکشنری در پایتون

متدها

- `clear`
- `items`
- `keys`
- `values`
- `pop`
- `popitem`
- `update`
- `copy`
- `setdefault`

متد Clear دیکشنری

از متد clear برای پاک کردن تمامی آیتم‌های دیکشنری در پایتون استفاده می‌شود. در قطعه کد زیر، مثالی از نحوه کاربرد این تابع ارائه شده است

```
1 d = {'a': 10, 'b': 20, 'c': 30}
2 print(d)
3
4 d.clear()
5 print(d)
```

خروجی قطعه کد فوق در ادامه ملاحظه می‌شود.

```
{'a': 10, 'b': 20, 'c': 30}
{}
```

متد Items دیکشنری

با استفاده از متد items می‌توان به لیستی از جفت کلید-مقدار دیکشنری دسترسی داشت. به عبارتی، خروجی این تابع لیستی از تاپل‌ها است که آیتم اول تاپل، کلید و آیتم دوم آن، مقدار کلید را مشخص می‌کند. در ادامه، مثالی از نحوه کاربرد این [متد در پایتون](#) ارائه شده است.

```
1 d = {'a': 10, 'b': 20, 'c': 30}
2
3 print(d)
4 print(list(d.items()))
5 print(list(d.items())[1][0])
6 print(list(d.items())[1][1])
```

در ادامه، خروجی قطعه کد فوق ملاحظه می‌شود.

```
{'a': 10, 'b': 20, 'c': 30}
[('a', 10), ('b', 20), ('c', 30)]
'b'
20
```

متد Keys دیکشنری

با استفاده از متد `keys` می‌توان به لیستی از کلیدهای دیکشنری در پایتون دسترسی داشت. در ادامه، مثالی از نحوه کاربرد این متد در پایتون ارائه شده است.

```
1 d = {'a': 10, 'b': 20, 'c': 30}
2 print(d)
3 print(list(d.keys()))
```

خروجی قطعه کد فوق در ادامه ملاحظه می‌شود.

```
{'a': 10, 'b': 20, 'c': 30}
['a', 'b', 'c']
```

متد Values دیکشنری

با استفاده از متد `values` می‌توان به لیستی از مقادیر دیکشنری در پایتون دسترسی داشت. در ادامه، مثالی از نحوه کاربرد این متد در پایتون ارائه شده است.

```
1 d = {'a': 10, 'b': 20, 'c': 30}
2 print(d)
3 print(list(d.values()))
```

خروجی قطعه کد فوق در ادامه ملاحظه می‌شود.

```
{'a': 10, 'b': 20, 'c': 30}
[10, 20, 30]
```

متد Pop دیکشنری

با استفاده از [دستور pop در پایتون](#) می توان کلیدی را از دیکشنری پایتون حذف کرد و مقدار آن را پیش از حذف کلید، در خروجی بازگرداند. در ادامه مثالی از کاربرد این متد در پایتون ملاحظه می شود.

```
1 d = {'a': 10, 'b': 20, 'c': 30}
2
3 print(d.pop('b'))
4 print(d)
```

خروجی دستورات بالا به صورت زیر است:

```
20
{'a': 10, 'c': 30}
```

```
1 d = {'a': 10, 'b': 20, 'c': 30}
2
3 print(d.pop('z', -1))
4 print(d)
```

خروجی قطعه کد فوق در ادامه ملاحظه می شود.

```
-1
{'a': 10, 'b': 20, 'c': 30}
```

متد Update دیکشنری

از متد update به منظور ادغام کردن دو دیکشنری یا لیستی از تاپل‌ها با دیکشنری استفاده می‌شود. در ادامه، نحوه استفاده از این متد در پایتون ارائه شده است.

```
1 d1 = {'a': 10, 'b': 20, 'c': 30}
2 d2 = {'b': 200, 'd': 400}
3 d1.update(d2)
4 print(d1)
5
6
7 d3 = {'a': 10, 'b': 20, 'c': 30}
8 d3.update([('b', 200), ('d', 400)])
9 print(d3)
10
11
12 d4 = {'a': 10, 'b': 20, 'c': 30}
13 d4.update(b=200, d=400)
14 print(d4)
```

خروجی قطعه کد فوق در ادامه ملاحظه می‌شود.

```
{'a': 10, 'b': 200, 'c': 30, 'd': 400}
{'a': 10, 'b': 200, 'c': 30, 'd': 400}
{'a': 10, 'b': 200, 'c': 30, 'd': 400}
```


دیکشنری تو در تو در پایتون چیست ؟

در پایتون می‌توان دیکشنری‌های تو در تو ایجاد کرد. به عبارتی، کلیدهای دیکشنری می‌توانند مقادیری از نوع ساختار داده دیکشنری داشته باشند که هر کدام از این دیکشنری‌ها دارای کلیدها و مقادیر هستند.

```
1 people = {1: {'name': 'John', 'age': '27', 'sex': 'Male'},  
2           2: {'name': 'Marie', 'age': '22', 'sex': 'Female'}}  
3  
4 print(people)
```

خروجی قطعه کد فوق در ادامه ملاحظه می‌شود.

```
{1: {'name': 'John', 'age': '27', 'sex': 'Male'}, 2: {'name': 'Marie', 'age': '22',  
'sex': 'Female'}}
```

دسترسی به مقادیر دیکشنری‌های تو در تو در پایتون

به منظور دسترسی به آیتمی خاص در دیکشنری‌های تو در تو می‌توان از علامت براکت ([]) به صورت زیر استفاده کرد:

```
1 people = {1: {'name': 'John', 'age': '27', 'sex': 'Male'},  
2           2: {'name': 'Marie', 'age': '22', 'sex': 'Female'}}  
3  
4 print(people[1]['name'])  
5 print(people[1]['age'])  
6 print(people[1]['sex'])
```

```
John  
27  
Male
```

اضافه کردن آیتم جدید به دیکشنری های تو در تو در پایتون

به منظور اضافه کردن مقداری خاص در دیکشنری های تو در تو نیز از علامت براکت ([]) استفاده می شود. در قطعه کد زیر، نحوه اضافه کردن مقادیر جدید به دیکشنری های تو در تو ملاحظه می شود.

```
1 people = {1: {'name': 'John', 'age': '27', 'sex': 'Male'},  
2           2: {'name': 'Marie', 'age': '22', 'sex': 'Female'}}  
3  
4 people[3] = {}  
5  
6 people[3]['name'] = 'Luna'  
7 people[3]['age'] = '24'  
8 people[3]['sex'] = 'Female'  
9 people[3]['married'] = 'No'  
10  
11 print(people[3])
```

خروجی دستورات فوق به صورت زیر است:

```
{'name': 'Luna', 'age': '24', 'sex': 'Female', 'married': 'No'}
```